

LineFollowerPipeline

Pure-Java, zero-dependency line-following vision processor

Single-file library · Java 8+ · No external dependencies

What This Library Does

`LineFollowerPipeline.java` is a pure-Java, zero-dependency line-following vision processor. You feed it raw grayscale camera frames, and it returns two integer control values your robot can act on directly — a steering direction and a speed/tracking signal.

No OpenCV, no Android APIs, and no native libraries are required.

Requirements

- Java 8 or higher
- One file only: `LineFollowerPipeline.java`
- No external dependencies

Minimal Integration (4 Steps)

1 Create one instance (keep it alive for the session)

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();
```

2 Calibrate — feed frames until it returns true

```
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    grayscaleBytes = camera.getGrayscaleFrame();
}
```

3 Call processFrame() on every camera frame

```
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

4 Read the two output values

```
int steering = result[0]; // use for direction control
int tracking = result[1]; // use for speed control
```

Calibration is required. `processFrame()` must not be called until `calibrate()` has returned `true`. The robot must be correctly placed on the line while calibrating. See **Calibration** below.

Input Format

```
public int[] processFrame(byte[] rawGrayscaleData, int width, int height)
```

Parameter	Type	Meaning
rawGrayscaleData	byte[]	Packed grayscale pixel data
width	int	Frame width in pixels
height	int	Frame height in pixels

Important: The array must be packed with no row padding. If your camera delivers padded rows (e.g. Android CameraX YUV planes), strip the padding before passing the data in.

Output Format

`processFrame()` returns a reused `int[2]` array. Copy the values immediately if you need to keep them — the same array is overwritten on the next call.

Index	Name	Range	Meaning
result[0]	Steering	-100 ... +100	0 = line is centred. Negative = line is to the left, turn left. Positive = line is to the right, turn right.
result[1]	Tracking	0 ... 100	100 = straight path ahead, full speed. Drops toward 0 as a turn or curve is detected. 0 = hard turn or line lost, brake/stop.

Typical robot wiring:

```
robot.setSteeringAngle(result[0]); // direction
robot.setSpeed(result[1]);       // speed — naturally slows before turns
```

Optional Configuration

If you need to tune the pipeline for your specific camera mount, line colour, or robot body size, create a `Config` object and pass it before the first frame:

```
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
// --- apply your settings here ---
pipeline.configure(cfg); // must be called BEFORE the first processFrame()
```

Line Colour

Field	Type	Default	Description
trackIsBright	boolean	true	true = white line on dark floor. false = dark line on light floor.
thresholdMargin	int	0	Bias added to the auto threshold. Increase if false positives occur. Decrease if the line is being missed.
bottomIgnoreFrac	float	0.15f	Fraction of the frame bottom to ignore — use this to exclude the robot's own body from the image. Increase until the chassis disappears from the scan.

Scan Strip (H-ROI — Near Line / Steering)

Field	Type	Default	Description
hRoiCenterYFrac	float	0.85f	Vertical position of the near scan strip as a fraction of frame height. 0.85 = 85% down from the top (near the bottom).
hRoiThickPx	int	24	Height of the scan strip in pixels.

Look-Ahead Column (V-ROI — Turn Preview / Tracking)

Field	Type	Default	Description
vRoiHalfWidthPx	int	80	Half-width in pixels of the look-ahead column. Increase on tight curves so the line stays inside the scan window.
vRoiHeightFrac	float	0.5f	How far ahead to look, as a fraction of frame height. 0.5 = half a frame ahead. Larger = more warning of far turns but noisier.
vRoiAdaptiveThreshold	boolean	true	Gives the look-ahead its own Otsu threshold. Recommended — the far line is dimmer than the near line and benefits from separate calibration.

Track Memory

Field	Type	Default	Description
trackMemoryMinGatePx	int	48	Minimum gate width in pixels regardless of line width.
trackLostGraceFrames	int	8	Frames to wait after losing the line before searching the full frame for a new one. At 30 fps, 8 ≈ 0.25 seconds.

Sensitivity

Field	Type	Default	Description
minWeightFrac	float	0.01f	Minimum fraction of scan strip pixels that must be "track" to trust the detection. Raise to reject weak signals.
trackOffsetFullFrac	float	1.0f	How far the look-ahead line must lean (as a fraction of half-frame width) to push Tracking to 0. Lower (e.g. 0.6) makes turns read earlier.

Calibration

Field	Type	Default	Description
calibFrames	int	10	Number of solid detections <code>calibrate()</code> collects before locking the straight-ahead zero reference (it takes their median). Assumes the robot is correctly placed on the line. Increase for more robust calibration.

Calibration

Before calling `processFrame()` you must first calibrate the pipeline. Place the robot correctly on the line, then call `calibrate()` once per camera frame until it returns `true` :

```
// Phase 1 – Calibration (at startup, or when the user presses calibrate)
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    // keep feeding frames until calibration is complete
    grayscaleBytes = camera.getGrayscaleFrame();
}

// Phase 2 – Normal operation
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

`calibrate()` collects a set number of solid line detections (default: **10 frames**, controlled by `Config.calibFrames`), takes their **median** position, and locks it as the straight-ahead zero reference. It returns `true` once this is done.

If the robot needs to re-calibrate mid-run:

```
pipeline.requestRecalibration();
// then feed frames to calibrate() again before resuming processFrame()
```

Runtime Utility Methods

These can be called at any time after the first `processFrame()` :

```
pipeline.getFramesPerSecond() // frames processed in the last completed second
pipeline.getTotalFrames()     // cumulative frame count since init
pipeline.getElapsedSeconds()  // wall-clock seconds since first frame
pipeline.isCalibrated()       // true once the straight-ahead reference is captured
pipeline.requestRecalibration() // resets calibration – must call calibrate() again before processFrame()
pipeline.release()            // shut down and reset
```

Full Example

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();

// Optional tuning
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
cfg.trackIsBright    = true;    // white line on black floor
cfg.cameraTiltDeg    = 45f;     // phone mount angle
cfg.bottomIgnoreFrac = 0.20f;   // hide robot body from scan
cfg.trackMemory      = true;    // lock onto line, ignore reflections
pipeline.configure(cfg);

// Step 1 – Calibrate (robot must be correctly placed on the line)
byte[] frame = camera.getGrayscaleFrame();
while (!pipeline.calibrate(frame, 1280, 720)) {
    frame = camera.getGrayscaleFrame();
}

// Step 2 – Normal operation
while (running) {
    frame = camera.getGrayscaleFrame();
    int[] result = pipeline.processFrame(frame, 1280, 720);

    robot.setSteeringAngle(result[0]); // -100 ... +100
    robot.setSpeed(result[1]);         //      0 ... 100
}
```

Behaviour Reference

Situation	Steering	Tracking
Straight line, centred	~0	~100
Line drifting to one side	± small	high
Curve approaching (line leaning)	varies	dropping toward 0
Hard turn	±100	0–30
Line completely lost	±100	0

Notes

- The pipeline is **not thread-safe**. Call `processFrame()` from a single thread only.
- The returned `int[]` is reused on every call. Copy the values if you need them past the next frame.
- Calibration assumes the robot is correctly placed on the line while `calibrate()` is running. If it is not, call `requestRecalibration()` once it is repositioned, then calibrate again.